

# 12: Rust Senior Interview Guide, Part 2: Errors, Concurrency, CLI/SDK, Cloud

The first Rust guide covered data structures and types. This second guide covers the skills that production Rust services and tools depend on, and that GPU platform roles ask about directly: typed error handling, concurrency with threads and async tasks, the design of an SDK and a command-line tool, and the cloud and Kubernetes context those tools run in.

## This chapter covers

- Choosing between `Option`, `Result`, typed errors with `thiserror`, and `anyhow`
- How the `?` operator converts errors, and when to panic instead
- Threads and channels for CPU-bound work, and `tokio` for I/O-bound work
- Timeouts, retries with backoff and jitter, graceful shutdown, and blocking calls in async code
- The structure of a Rust SDK and a CLI built on it
- REST, retry, container, Kubernetes, and CI concepts for a GPU cloud service

## Contents

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 1. Error handling.....                                                     | 3  |
| 1.1 <code>Option&lt;T&gt;</code> and <code>Result&lt;T, E&gt;</code> ..... | 3  |
| 1.2 Typed errors for libraries with <code>thiserror</code> .....           | 3  |
| 1.3 Contextual errors for applications with <code>anyhow</code> .....      | 4  |
| 1.4 The <code>?</code> operator and <code>From</code> .....                | 5  |
| 1.5 Functions with several error sources .....                             | 6  |
| 1.6 Panics and errors .....                                                | 6  |
| 2. Concurrency and async .....                                             | 7  |
| 2.1 Threads and channels for CPU-bound work .....                          | 7  |
| 2.2 Async tasks with <code>tokio</code> for I/O-bound work.....            | 8  |
| 2.3 Timeouts .....                                                         | 8  |
| 2.4 Retries with exponential backoff and jitter .....                      | 9  |
| 2.5 Graceful shutdown.....                                                 | 11 |
| 2.6 Blocking work in async code .....                                      | 12 |
| 2.7 Common async mistakes.....                                             | 12 |
| 3. Building an SDK and a CLI in Rust .....                                 | 13 |
| 3.1 Crates .....                                                           | 13 |
| 3.2 <code>Cargo.toml</code> .....                                          | 13 |
| 3.3 The SDK: <code>src/lib.rs</code> .....                                 | 14 |
| 3.4 The CLI: <code>src/main.rs</code> .....                                | 17 |
| 3.5 SDK design rules .....                                                 | 19 |

|                                                      |    |
|------------------------------------------------------|----|
| 4. Cloud concepts for a GPU platform .....           | 20 |
| 4.1 The control plane .....                          | 20 |
| 4.2 REST methods and idempotency .....               | 20 |
| 4.3 Retries, circuit breakers, and rate limits ..... | 21 |
| 4.4 A multi-stage Dockerfile .....                   | 21 |
| 4.5 Kubernetes objects for a GPU service .....       | 22 |
| 4.6 CI for a Rust project .....                      | 23 |
| 5. Summary .....                                     | 24 |

# 1. Error handling

## 1.1 Option<T> and Result<T, E>

- Option<T> represents a value that may be absent, where absence is a normal outcome and needs no explanation, such as a lookup that finds nothing.
- Result<T, E> represents an operation that can fail, where the caller needs to know why.

```
fn find_region<'a>(regions: &'a [&str], target: &str) -> Option<&'a str> {
    regions.iter().copied().find(|r| *r == target)
}

fn allocate_gpu(quota: u32, requested: u32) -> Result<(), GpuError> {
    if requested > quota {
        return Err(GpuError::InsufficientQuota {
            quota,
            requested,
        });
    }
    Ok(())
}
```

Convert between them with Option::ok\_or (or ok\_or\_else) and Result::ok.

## 1.2 Typed errors for libraries with thiserror

A library should return an error enum with one variant per failure the caller might handle differently. The thiserror crate generates the boilerplate:

```

use std::time::Duration;

#[derive(Debug, thiserror::Error)]
pub enum GpuCloudError {
    #[error("invalid request: {0}")]
    InvalidRequest(String),

    #[error("authentication failed")]
    Unauthorized,

    #[error("rate limited; retry after {retry_after:?}")]
    RateLimited { retry_after: Duration },

    #[error("resource {resource} not found")]
    NotFound { resource: String },

    #[error("server error {status}: {message}")]
    Server { status: u16, message: String },

    #[error("network error: {0}")]
    Network(#[from] std::io::Error),
}

impl GpuCloudError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            GpuCloudError::RateLimited { .. }
            | GpuCloudError::Server { status: 500..=599, .. }
            | GpuCloudError::Network(_)
        )
    }
}

```

The derive generates:

- An implementation of Display, using the `#[error("...")]` message for each variant.
- An implementation of `std::error::Error`. A field marked `#[from]` or `#[source]` is reported as the error's source, so error-reporting tools can print the full chain.
- For each `#[from]` field, an implementation of `From<SourceError>`, so the `?` operator converts that error automatically.

The `is_retryable` method puts the retry decision next to the error definition, where it is easy to keep correct as variants are added.

### 1.3 Contextual errors for applications with anyhow

In an application, such as a CLI, most errors are reported to the user rather than handled by type. `anyhow::Result` accepts any error type and lets you add context as the error propagates:

```

use anyhow::{Context, Result};

fn main() -> Result<()> {
    let config = std::fs::read_to_string("config.yaml")
        .context("failed to read config.yaml")?;
    println!("{}", config);
    Ok(())
}

```

If the file is missing, the program prints both messages: failed to read config.yaml and the underlying No such file or directory error.

Do not return `anyhow::Error` from a library's public API. Callers cannot match on it to handle specific failures. Libraries expose typed errors, and applications wrap them with `context`.

## 1.4 The `?` operator and `From`

`?` returns early with the error when a `Result` is `Err`. Before returning, it calls `From::from` to convert the error into the function's error type:

```

fn load_and_parse(path: &str) -> Result<u32, Box<dyn std::error::Error>> {
    let raw = std::fs::read_to_string(path)?;           // io::Error -> Box<dyn Error>
    let count = raw.trim().parse::<u32>()?;             // ParseIntError -> Box<dyn Error>
    Ok(count)
}

```

With a custom error enum, `#[from]` provides the `From` implementations:

```

#[derive(Debug, thiserror::Error)]
enum ConfigError {
    #[error("io error: {0}")]
    Io(#[from] std::io::Error),

    #[error("invalid gpu count: {0}")]
    Parse(#[from] std::num::ParseIntError),
}

fn parse_gpu_count(path: &str) -> Result<u32, ConfigError> {
    let raw = std::fs::read_to_string(path)?; // auto From<io::Error>
    let count = raw.trim().parse::<u32>()?;   // auto From<ParseIntError>
    Ok(count)
}

```

## 1.5 Functions with several error sources

| Approach                                | Advantages                      | Disadvantages                                         | Use in                      |
|-----------------------------------------|---------------------------------|-------------------------------------------------------|-----------------------------|
| <code>Box&lt;dyn Error&gt;</code>       | No extra types to define        | Callers cannot match on the cause without downcasting | Examples and small programs |
| An error enum with <code>#[from]</code> | Callers can match on each cause | More code to maintain                                 | Libraries                   |
| <code>anyhow::Error</code>              | Easy context and propagation    | Callers cannot match on the cause without downcasting | Applications                |

You can also convert errors explicitly with `map_err` when a source error should become a specific variant. The following fragment assumes `Workload` and `ParseWorkloadError` types defined elsewhere:

```
fn parse_workload(raw: &str) -> Result<Workload, ParseWorkloadError> {
    // return typed error variants
    if raw.is_empty() {
        return Err(ParseWorkloadError::Empty);
    }
    let fields: Vec<&str> = raw.split(',').collect();
    let gpus: u32 = fields[1].parse().map_err(ParseWorkloadError::BadGpuCount)?;
    Ok(Workload { image: fields[0].into(), gpus })
}
```

**Warning:** `fields[1]` panics if the input contains no comma. A parser for untrusted input should use `fields.get(1)` and return an error variant when the field is missing.

## 1.6 Panics and errors

- **Panic for bugs**, meaning violated invariants that indicate a mistake in the program. Do not panic because of bad user input or a failed network call; return an error.
- `unwrap()` **is acceptable** in tests, in examples, and where the code has just established that the value is present.
- **Prefer** `expect("...")` where you rely on an invariant, with a message that states it, such as `expect("config was validated at startup")`.
- **Public library functions return** `Result` for any failure a caller could reasonably encounter.

## 2. Concurrency and async

Use operating system threads for CPU-bound work, and async tasks for I/O-bound work where a program waits on many connections or timers at once.

### 2.1 Threads and channels for CPU-bound work

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();
    let mut handles = Vec::new();

    for worker_id in 0..4 {
        let tx = tx.clone();
        handles.push(thread::spawn(move || {
            for job_id in 0..10 {
                tx.send((worker_id, job_id)).unwrap();
            }
        }));
    }
    drop(tx); // close channel after all senders

    let mut received = 0;
    while let Ok((worker_id, job_id)) = rx.recv() {
        println!("worker {worker_id} finished job {job_id}");
        received += 1;
    }
    assert_eq!(received, 40);

    for handle in handles {
        handle.join().unwrap();
    }
}
```

Each worker gets its own clone of the sender. `rx.recv()` returns `Err` only when every sender has been dropped. The workers' clones are dropped when the workers finish, but the original `tx` would keep the channel open forever, so the example drops it explicitly before the receive loop. Forgetting that `drop` is a common cause of a program that never exits.

## 2.2 Async tasks with tokio for I/O-bound work

```
use std::time::Duration;
use tokio::time::sleep;

#[tokio::main]
async fn main() {
    let mut tasks = Vec::new();

    for i in 0..10 {
        tasks.push(tokio::spawn(async move {
            sleep(Duration::from_millis(100)).await;
            i * i
        }));
    }

    let mut total = 0u64;
    for task in tasks {
        total += task.await.unwrap();
    }
    println!("total = {total}");
}
```

The ten tasks sleep concurrently, so the program takes about 100 milliseconds rather than one second. `tokio::spawn` returns a `JoinHandle`; awaiting it returns `Err` if the task panicked or was cancelled.

## 2.3 Timeouts

`tokio::time::timeout` wraps a future and returns `Err(Elapsed)` if the future does not finish in time. The inner future is dropped at that point, which cancels it.

```
use std::time::Duration;
use tokio::time::{sleep, timeout};

#[tokio::main]
async fn main() {
    let slow = async {
        sleep(Duration::from_secs(10)).await;
        42
    };

    match timeout(Duration::from_millis(50), slow).await {
        Ok(value) => println!("got {value}"),
        Err(_) => println!("timed out"),
    }
}
```

To wait for whichever of several futures finishes first, use `tokio::select!`, shown in section 2.5.

## 2.4 Retries with exponential backoff and jitter

A retry policy waits longer after each failure (exponential backoff), caps the wait, and chooses a random delay up to that cap (*full jitter*). The randomness spreads out the retries of many clients that failed at the same moment, so they do not overload the server again together.

```

use std::time::Duration;

pub struct RetryPolicy {
    pub max_attempts: u32,
    pub base_delay: Duration,
    pub max_delay: Duration,
}

impl RetryPolicy {
    pub fn delay_for_attempt(&self, attempt: u32) -> Duration {
        // Exponential: base * 2^attempt, capped.
        let exp = self.base_delay.saturating_mul(1u32 << attempt.min(10));
        let capped = exp.min(self.max_delay);
        // Full jitter: random between zero and capped avoids thundering herd.
        let nanos = capped.as_nanos();
        let jittered = if nanos == 0 {
            0
        } else {
            rand_simple(nanos)
        };
        Duration::from_nanos(jittered)
    }
}

fn rand_simple(max_nanos: u128) -> u64 {
    // A deterministic stand-in for a jitter source; production code uses `rand`.
    use std::time::{SystemTime, UNIX_EPOCH};
    let seed = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_nanos();
    ((seed % max_nanos) as u64).max(1)
}

async fn call_with_retry<F, T, E>(policy: &RetryPolicy, mut f: F) -> Result<T, E>
where
    F: FnMut() -> Result<T, E>,
    E: IsRetryable,
{
    let mut last_error = None;
    for attempt in 0..policy.max_attempts {
        match f() {
            Ok(value) => return Ok(value),
            Err(error) if error.is_retryable() => {
                last_error = Some(error);
                tokio::time::sleep(policy.delay_for_attempt(attempt)).await;
            }
            Err(error) => return Err(error),
        }
    }
    Err(last_error.expect("max_attempts must be > 0"))
}

trait IsRetryable {
    fn is_retryable(&self) -> bool;
}

```

Points to discuss about this implementation:

- `1u32 << attempt.min(10)` limits the multiplier to 1,024, which prevents the shift from overflowing, and `saturating_mul` prevents the multiplication from overflowing.
- `rand_simple` **is not a good random source**. The code comment calls it deterministic, but it derives the value from the current time; clients that fail together read similar clock values and choose similar delays. Use the `rand` crate in real code.
- **The loop sleeps after the last failed attempt** before returning the error, which adds a delay that serves no purpose. Check whether another attempt remains before sleeping.
- `f` **is a synchronous closure**. An SDK would take a closure that returns a future, so the operation being retried can itself be async.
- **The policy ignores** `Retry-After`. When the server supplies a delay, use it instead of the computed backoff.

The repository's `rust-interview-lab/src/problems/retry.rs` contains a tested retry implementation.

## 2.5 Graceful shutdown

A service should stop accepting work when it receives a shutdown signal, let current work finish, and then exit. A watch channel broadcasts the shutdown request to every task, and `tokio::select!` waits for either work or shutdown:

```
use tokio::signal;
use tokio::sync::watch;
use tokio::time::{interval, Duration};

#[tokio::main]
async fn main() {
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);
    let worker = tokio::spawn(async move {
        let mut ticker = interval(Duration::from_secs(1));
        loop {
            tokio::select! {
                _ = ticker.tick() => println!("working..."),
                _ = shutdown_rx.changed() => {
                    println!("shutdown requested");
                    break;
                }
            }
        }
    });

    // In a real service this would wait for SIGTERM/SIGINT.
    signal::ctrl_c().await.ok();

    shutdown_tx.send(true).ok();
    worker.await.unwrap();
}
```

In Kubernetes, the kubelet sends SIGTERM to stop a container, so a service there must also handle SIGTERM, with `tokio::signal::unix::signal(SignalKind::terminate())`. `tokio_util::sync::CancellationToken` is an alternative to the watch channel, designed specifically for cancellation.

## 2.6 Blocking work in async code

The tokio runtime runs many tasks on a few threads. A task that blocks, for example with `std::fs::read_to_string` or a long computation, stops every other task on that thread from making progress. Move blocking work to a separate thread pool with `tokio::task::spawn_blocking`:

```
async fn read_big_file(path: String) -> std::io::Result<String> {
    tokio::task::spawn_blocking(move || std::fs::read_to_string(path))
        .await
        .map_err(|join_error| std::io::Error::other(join_error))?
}
```

`spawn_blocking` returns a `JoinHandle`. Awaiting it yields `Result<io::Result<String>, JoinError>`. The `map_err` converts the `JoinError` into an `io::Error`, ? returns it if the task failed, and the remaining `io::Result<String>` is the function's return value. For file I/O specifically, `tokio::fs` performs this wrapping for you.

## 2.7 Common async mistakes

- **Expecting an `async fn` to start by itself.** Calling it creates a future; nothing runs until the future is awaited or spawned.
- **Holding a non-Send value across `.await` in a spawned task.** `tokio::spawn` requires the future to be `Send + 'static`. An `Rc` or a `std::sync::MutexGuard` that is alive at an `.await` makes the future non-Send, and the code does not compile.
- **Holding a lock across `.await`.** Release a `std::sync::Mutex` guard before awaiting. If a lock must be held across an await point, use `tokio::sync::Mutex`.
- **Blocking the runtime.** Use `spawn_blocking` for blocking calls and CPU-heavy work.

## 3. Building an SDK and a CLI in Rust

### 3.1 Crates

| Need                      | Crate                                                       |
|---------------------------|-------------------------------------------------------------|
| Command-line parsing      | clap, with the derive feature                               |
| Async HTTP                | request, with the json feature and rustls-tls or native TLS |
| JSON                      | serde and serde_json                                        |
| Async runtime             | tokio, with macros, rt-multi-thread, time, and signal       |
| Library errors            | thiserror                                                   |
| Application errors        | anyhow                                                      |
| Configuration directories | directories                                                 |
| Table output              | comfy-table                                                 |
| Logging                   | tracing and tracing-subscriber                              |

### 3.2 Cargo.toml

```
[package]
name = "gpucloud-cli"
version = "0.1.0"
edition = "2024"

[dependencies]
anyhow = "1"
clap = { version = "4", features = ["derive"] }
request = { version = "0.12", features = ["json", "rustls-tls"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
thiserror = "2"
tokio = { version = "1", features = ["macros", "rt-multi-thread", "time", "signal"] }
tracing = "0.1"
tracing-subscriber = "0.3"
```

The package contains a library (`src/lib.rs`), which is the SDK, and a binary (`src/main.rs`), which is the CLI. The CLI uses the SDK through its public API, the same way any other program would.

### 3.3 The SDK: `src/lib.rs`

```

use serde::{Deserialize, Serialize};
use std::time::Duration;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct GpuWorkload {
    pub id: String,
    pub name: String,
    pub gpu_type: String,
    pub gpu_count: u32,
    pub status: WorkloadStatus,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
#[serde(rename_all = "snake_case")]
pub enum WorkloadStatus {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed,
}

#[derive(Debug, thiserror::Error)]
pub enum SdkError {
    #[error("HTTP {status}: {message}")]
    Http { status: u16, message: String },

    #[error("rate limited; retry after {retry_after:?}")]
    RateLimited { retry_after: Duration },

    #[error("server returned invalid JSON: {0}")]
    Json(#[from] serde_json::Error),

    #[error("network error: {0}")]
    Network(#[from] request::Error),
}

impl SdkError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            SdkError::RateLimited { .. }
            | SdkError::Http { status: 500..=599, .. }
            | SdkError::Network(_)
        )
    }
}

#[derive(Debug, Clone)]
pub struct ClientConfig {
    pub base_url: String,
    pub api_key: String,
    pub timeout: Duration,
    pub max_retries: u32,
}

pub struct GpuCloudClient {

```

```

    http: request::Client,
    config: ClientConfig,
}

impl GpuCloudClient {
    pub fn new(config: ClientConfig) -> anyhow::Result<Self> {
        let http = request::Client::builder()
            .timeout(config.timeout)
            .build()?;
        Ok(Self { http, config })
    }

    pub async fn list_workloads(&self) -> Result<Vec<GpuWorkload>, SdkError> {
        let url = format!("{}/v1/gpu-workloads", self.config.base_url);
        let response = self
            .http
            .get(url)
            .bearer_auth(&self.config.api_key)
            .send()
            .await?;

        if response.status().is_success() {
            return Ok(response.json().await?);
        }

        let status = response.status().as_u16();
        let message = response.text().await.unwrap_or_default();
        Err(SdkError::Http { status, message })
    }
}

```

Notes on the design:

- **Models are plain data types** with `Serialize` and `Deserialize`. `rename_all = "snake_case"` maps `WorkloadStatus::Running` to the JSON string `"running"`.
- **The client holds one** `request::Client`, which maintains a connection pool. Create the SDK client once and reuse it.
- `list_workloads` **checks the status code before parsing**, so an error response becomes `SdkError::Http` with the server's message rather than a JSON parse error.
- `GpuCloudClient::new` **returns** `anyhow::Result`. For consistency with the rest of the SDK's typed errors, a published library would return `Result<Self, SdkError>`.
- `max_retries` **is not used yet**. A complete client would apply the retry policy from section 2.4 to retryable errors.

### 3.4 The CLI: `src/main.rs`

```

use anyhow::{Context, Result};
use clap::{Parser, Subcommand};
use gpucloud_cli::{ClientConfig, GpuCloudClient};
use std::time::Duration;

#[derive(Parser)]
#[command(name = "gpucloud", version, about = "GPU cloud CLI")]
struct Cli {
    #[arg(long, env = "GPU_CLOUD_API_KEY")]
    api_key: String,

    #[arg(long, default_value = "https://api.gpucloud.example.com")]
    base_url: String,

    #[command(subcommand)]
    command: Command,
}

#[derive(Subcommand)]
enum Command {
    /// List GPU workloads
    List,
    /// Get one workload
    Get { id: String },
    /// Create a workload
    Create {
        name: String,
        #[arg(long)]
        gpu_type: String,
        #[arg(long, default_value_t = 1)]
        count: u32,
    },
}

#[tokio::main]
async fn main() -> Result<> {
    let cli = Cli::parse();

    let client = GpuCloudClient::new(ClientConfig {
        base_url: cli.base_url,
        api_key: cli.api_key,
        timeout: Duration::from_secs(10),
        max_retries: 3,
    })
    .context("failed to create client")?;

    match cli.command {
        Command::List => {
            let workloads = client.list_workloads().await?;
            for workload in workloads {
                println!("{:} {:} {}x{:}", workload.id, workload.status_label(), workload.gpu_type,
workload.gpu_count);
            }
        }
        Command::Get { id } => {
            let workload = client.get_workload(&id).await?;
            println!("{:?}", workload);
        }
    }
}

```

```

    Command::Create { name, gpu_type, count } => {
        let workload = client.create_workload(&name, &gpu_type, count).await?;
        println!("created {}", workload.id);
    }
}

Ok(())
}

```

The clap derive API builds the parser from the types. Doc comments on the variants become the help text for each subcommand, and `env = "GPUCLLOUD_API_KEY"` reads the key from the environment when the flag is omitted, which keeps secrets out of the shell history.

**Note:** This file does not compile against the `lib.rs` in section 3.3 as shown. It calls `status_label()`, `get_workload`, and `create_workload`, which that listing does not define. Each would follow the pattern of `list_workloads`: build the URL, send the request with authentication, and map the response to a typed result or an `SdkError`.

### 3.5 SDK design rules

1. **The client owns one HTTP client** and reuses its connections.
2. **Public methods take typed arguments and return** `Result<T, SdkError>`.
3. **Retries apply only to idempotent requests**, or to requests sent with an `Idempotency-Key`.
4. **Retry-After is honored** when the server sends it.
5. **Pagination is exposed as an iterator or stream**, so callers never handle page tokens.
6. **Library code propagates errors with** `?` and does not call `unwrap()`.
7. **Authentication and token refresh are handled in one place**, inside the client.

## 4. Cloud concepts for a GPU platform

06-system-design-cloud-gpu.md and 07-cli-sdk-rest-concurrency.md cover these topics in more depth. This section summarizes what a Rust SDK or service needs to account for.

### 4.1 The control plane

```
CLI/SDK -> API Gateway -> Auth/Quota/Placement -> Workflow -> Kubernetes
                                                    -> GpuWorkload CR
                                                    -> Deployment/Job
                                                    -> Device plugin
                                                    -> GPU node
```

| Concept                 | What it means for the service                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------------|
| Resources               | GPU instances, GPU workloads, reservations, snapshots, and node pools                                |
| Asynchronous operations | Creates return 202 Accepted with a Location, and clients poll the operation                          |
| Idempotency             | Clients send an Idempotency-Key, and the server does not create a duplicate                          |
| Quota                   | Reservations with a timeout in the control plane, in addition to Kubernetes ResourceQuota            |
| State machine           | Pending, Provisioning, Running, Terminating, Terminated, with Failed reachable from any active state |
| Billing                 | Metered from GPU usage data such as DCGM metrics, not only from creation and deletion times          |

### 4.2 REST methods and idempotency

A method is *idempotent* if sending the same request several times has the same effect as sending it once. Clients can safely retry idempotent requests.

| Method | Path                   | Idempotent                                                                             |
|--------|------------------------|----------------------------------------------------------------------------------------|
| GET    | /v1/gpu-workloads      | Yes                                                                                    |
| POST   | /v1/gpu-workloads      | No, unless the client sends an idempotency key                                         |
| GET    | /v1/gpu-workloads/{id} | Yes                                                                                    |
| PUT    | /v1/gpu-workloads/{id} | Yes: it replaces the resource with the same content                                    |
| PATCH  | /v1/gpu-workloads/{id} | Not by definition; a patch that sets fields is, but one that increments a value is not |
| DELETE | /v1/gpu-workloads/{id} | Yes: the resource is deleted, although a repeat may return 404                         |

List endpoints should use cursor-based pagination:

```
GET /v1/gpu-workloads?limit=50&page_token=abc
-> { "items": [...], "next_page_token": "def" }
```

A page token identifies a position in the list. With page numbers, an item created or deleted while a client is paging shifts the remaining items, and the client skips or repeats items. Cursors avoid that problem.

### 4.3 Retries, circuit breakers, and rate limits

- **Retry with exponential backoff and full jitter** (section 2.4).
- **Honor** Retry-After.
- **Retry only idempotent operations**, unless the API supports idempotency keys.
- **Use a circuit breaker** to stop sending requests to a dependency that keeps failing. After a number of consecutive failures, the breaker fails requests immediately for a period, then lets a few requests through to test whether the dependency has recovered.
- **Apply token-bucket rate limits** on both the client and the server.

### 4.4 A multi-stage Dockerfile

A multi-stage build compiles the program in an image that contains the Rust toolchain and copies only the binary into a small runtime image:

```

# Build stage
FROM rust:1-bookworm AS builder
WORKDIR /app
COPY . .
RUN cargo build --release

# Runtime stage
FROM debian:bookworm-slim
COPY --from=builder /app/target/release/gpucloud-cli /usr/local/bin/gpucloud
ENTRYPOINT ["gpucloud"]

```

Build on the same Debian release as the runtime image, as `rust:1-bookworm` and `debian:bookworm-slim` do here. A binary built against a newer C library than the runtime image provides fails to start. For a smaller image, build a static binary for the `x86_64-unknown-linux-musl` target and use `gcr.io/distroless/static` or `scratch`.

Exclude build output and repository metadata from the build context with `.dockerignore`:

```

target/
.git/
*.log

```

## 4.5 Kubernetes objects for a GPU service

| Object                       | Use                                                                            |
|------------------------------|--------------------------------------------------------------------------------|
| Pod                          | Requests <code>nvidia.com/gpu</code> in limits                                 |
| DaemonSet                    | Runs the device plugin, DCGM exporter, and driver components on every GPU node |
| Deployment                   | Runs stateless inference replicas                                              |
| StatefulSet                  | Runs training workers that need stable names or ranks                          |
| Job                          | Runs batch work to completion                                                  |
| Service                      | Gives a stable address to a set of Pods                                        |
| ConfigMap, Secret            | Provide configuration and credentials                                          |
| Custom resource and operator | Model domain objects and reconcile them into the objects above                 |
| ResourceQuota, LimitRange    | Limit each tenant's resource use                                               |
| NetworkPolicy                | Isolates tenants' network traffic                                              |

## 4.6 CI for a Rust project

```
name: CI
on: [push, pull_request]
jobs:
  rust:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - run: cargo fmt --check
      - run: cargo clippy -- -D warnings
      - run: cargo test
      - run: cargo build --release
```

`cargo clippy -- -D warnings` turns lint warnings into errors, so new warnings fail the build. A complete pipeline for this repository also runs `go test`, the Kind end-to-end test, a container image build and push, and a release job.

## 5. Summary

- Libraries return typed error enums, usually derived with `thiserror`. Applications use `anyhow` and add context.
- `?` propagates errors and converts them through `From`; `#[from]` generates the conversion.
- Use `threads` and `channels` for CPU-bound work and `tokio` tasks for I/O-bound work. Move blocking calls to `spawn_blocking`, and do not hold locks across `.await`.
- Retries need exponential backoff with a cap, real randomness for jitter, and respect for `Retry-After`.
- An SDK shares one HTTP client, uses typed models and errors, and hides pagination and authentication from callers.
- In Kubernetes, Pods request `nvidia.com/gpu`, the kubelet calls the device plugin's `Allocate`, and the container runtime makes the devices available.

Source: [12-rust-guide-part2-errors-concurrency-sdk.md](#) · [PDF](#) · [All study guides](#)